Регистрация на десктопе (Аэрофлот Бонус) в BPMN — простая версия

Oct 14, 2026 · @Cxrsed

Этап 1. Кто участвует и границы процесса

Я описал процесс, как человек регистрируется на сайте Аэрофлот Бонус с компьютера. В нём участвуют четыре участника (в BPMN это пулы):

- **Пользователь** — заполняет форму мышью и клавиатурой.
- **Веб-система Аэрофлот** — сам сайт. Внутри неё три дорожки: **UI/UX** (то, что видит пользователь), **AuthService** (сервер, который делает проверки) и **DataStore** (хранение данных).
- **SMS-шлюз** — внешняя система, которая присылает SMS с кодом.
- **База данных** — хранит пользователей, коды и согласия.

Границы. Старт — пользователь открыл страницу регистрации. Конец — он успешно зашёл в личный кабинет или получил финальное сообщение об ошибке.

Глоссарий (слова, которые встречаются в работе):

1. **Валидация на лету** — поле проверяется сразу, пока человек заполняет форму, а не после отправки.
2. **Автофокус** — курсор сам становится в первое поле, кликать не нужно.
3. **Tab-навигация** — переход между полями клавишей Tab.
4. **Подсвечивание поля** — поле с ошибкой выделяется (например, красной рамкой).
5. **Блокировка кнопки отправки** — кнопка «Отправить» не нажимается, пока в форме есть ошибки.

Этап 2. Основная диаграмма

Это общая схема регистрации. Читать её нужно слева направо: сверху — пользователь, ниже — сайт (три дорожки), потом SMS-шлюз и база данных. Файл для редактора BPMN — `registration_simple.bpmn`.

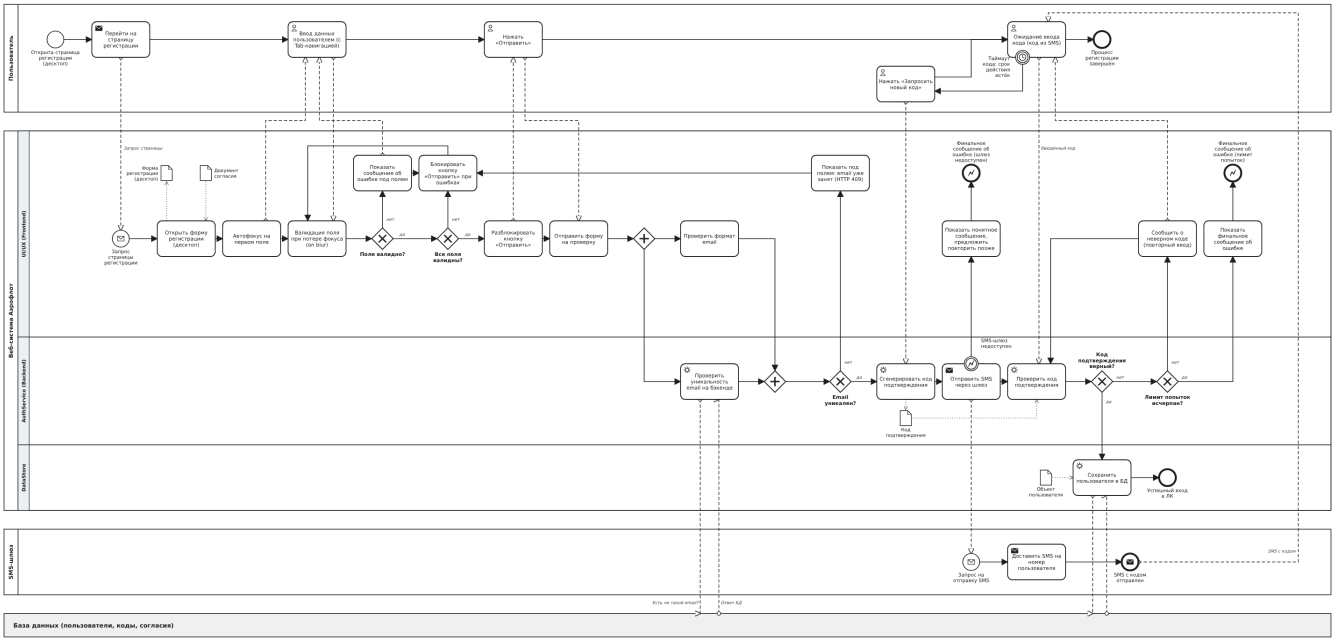
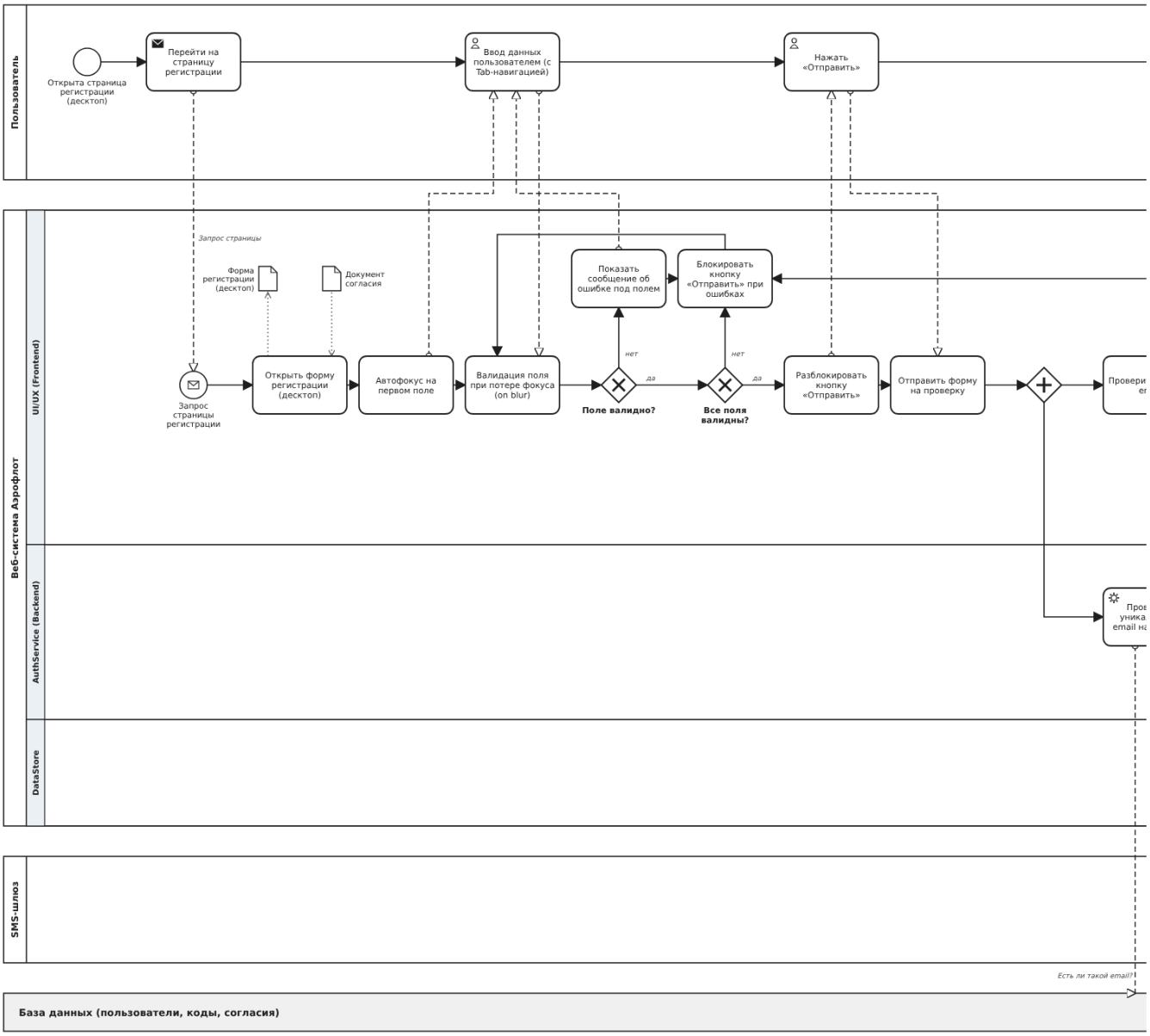
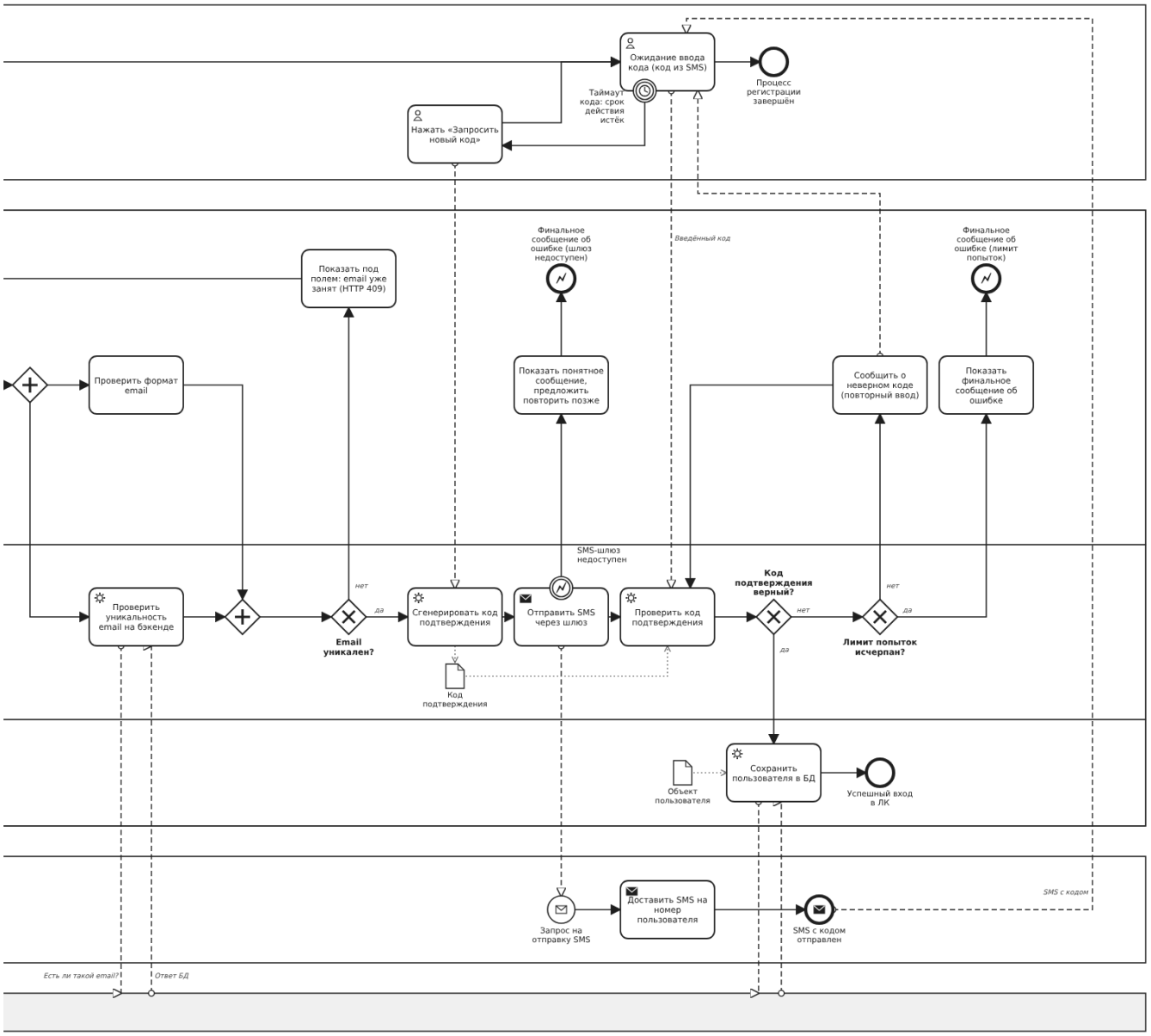


Диаграмма широкая, поэтому ниже она разрезана на две части, чтобы было лучше видно.


Пользователь
Открыта страница регистрации (десктоп)
Перейти на страницу регистрации
Ввод данных пользователем (с Tab-навигацией)
Нажать «Отправить»
Веб-система Аэрофлот
UI/UX (Frontend)
AuthService (Backend)
DataStore
Запрос страницы
Форма регистрации (десктоп)
Документ согласия
Показать сообщение об ошибке под полем
Блокировать кнопку «Отправить» при ошибках
Запрос страницы регистрации
Открыть форму регистрации (десктоп)
Автофокус на первом поле
Валидация поля при потере фокуса (on blur)
нет
да
Поле валидно?
нет
да
Все поля валидны?
Разблокировать кнопку «Отправить»
Отправить форму на проверку
SMS-шлюз
Есть ли такой email?
База данных (пользователи, коды, согласия)



Что нарисовано

- **Задачи** (прямоугольники) — действия: открыть форму, автофокус, ввод данных, проверка при потере фокуса (on blur), ошибка под полем, блокировка кнопки, проверка email, создание кода, отправка SMS, сохранение в БД.
- **Шлюзы** (ромбы). Ромб с крестиком — выбор «да или нет»: «Поле валидно?», «Все поля валидны?», «Email уникален?», «Код верный?». Ромб с плюсом — две проверки сразу вместе (формат email и его уникальность).
- **Сообщения** (пунктир) — обмен между участниками: сайт просит SMS-шлюз прислать код, база данных отвечает, есть ли такой email. Внутри сайта между дорожками стоят обычные стрелки, так правильно по правилам BPMN (пунктир бывает только между разными пулами).
- **Данные** (листочки): форма регистрации, объект пользователя, код подтверждения, документ согласия.

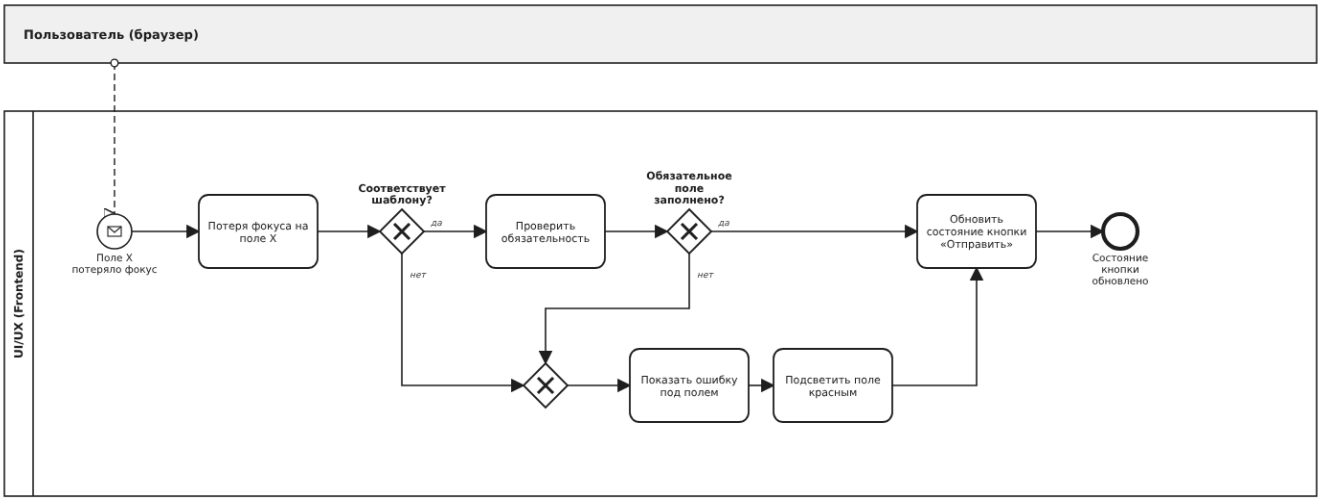
Пять сценариев

1. **Всё хорошо.** Поля заполнены верно, кнопка разблокирована, email свободен, приходит SMS, код введён верно, пользователь сохранён, вход в ЛК.
2. **Ошибка в поле** (неверный формат, спецсимволы, слишком коротко). Под полем появляется ошибка, кнопка блокируется, человек исправляет поле.
3. **Email уже есть** (ответ сервера 409). Под полем появляется сообщение, что email занят.
4. **Неверный код.** Можно ввести код ещё раз; когда попытки закончились, процесс заканчивается финальной ошибкой.
5. **Код просрочен.** Человек не ввёл код вовремя, срабатывает таймер, он нажимает «Запросить новый код» и получает новое SMS.

Этап 3. Две небольшие диаграммы

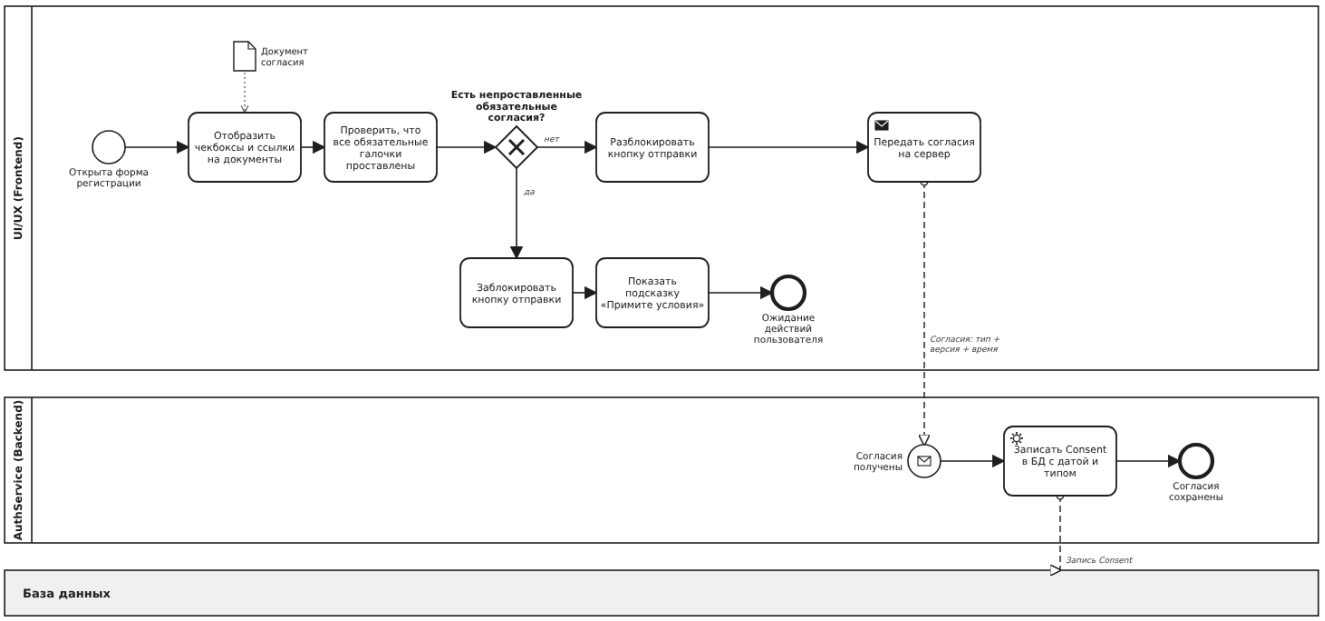
Два места из основной схемы я разбил подробнее — проверку полей и согласия. Файлы: `validation_simple.bpmn` и `consents_simple.bpmn`.

Проверка полей (on blur)



Когда человек уходит из поля, сайт проверяет, подходит ли значение по формату. Если нет — показывает ошибку под полем и подсвечивает поле красным. Если да — проверяет, что обязательное поле не пустое. В конце всегда обновляется состояние кнопки «Отправить». При нажатии «Отправить» проверка такая же, только для всех полей сразу.

Согласия



Сайт показывает галочки и ссылки на документы. Потом проверяет, все ли обязательные проставлены. Если нет — кнопка блокируется и появляется подсказка «Примите условия». Если всё отмечено — кнопка разблокируется, согласия уходят на сервер, и там их записывают в базу данных с датой и типом.

Этап 4. Требования и вывод

Нефункциональные требования

Это требования не к тому, «что делает» сайт, а ктому, «как» он работает:

1. **Доступность.** У каждого поля есть подпись, текст ошибки хорошо виден, форму можно заполнить только с клавиатуры.
2. **Скорость.** Проверка поля работает сразу, без перезагрузки страницы.
3. **Безопасность.** Пароль хранят не как есть, а в виде хэша (password_hash).
4. **Юридическое.** Согласия сохраняют в базе с датой и типом.
5. **Понятные ошибки.** Если SMS не работает, пользователь видит понятное сообщение.

Связь требований с диаграммой

Требование	Где на диаграмме
1. Доступность	Задачи «Открыть форму регистрации» (подписи полей), «Показать сообщение об ошибке под полем», «Ввод данных (с Tab-навигацией)»
2. Скорость	Задача «Валидация поля при потере фокуса (on blur)» и шлюз «Поле валидно?»
3. Безопасность	Задача «Сохранить пользователя в БД» (пароль как password_hash)
4. Юридическое	Задача «Записать Consent в БД с датой и типом» (диаграмма согласий)
5. Понятные ошибки	Граничное событие «SMS-шлюз недоступен» → задача «Показать понятное сообщение, предложить повторить позже»

Граничные события

Граничное событие — это кружок на краю задачи. Он срабатывает, если что-то пошло не так, и процесс уходит по запасному пути. На диаграмме два таких события:

1. **Таймаут ожидания кода** (часы на задаче «Ожидание ввода кода»). Если человек не ввёл код за отведённое время, он нажимает «Запросить новый код», и сайт создаёт новый код и шлёт новое SMS.
2. **Ошибка SMS-шлюза** (молния на задаче «Отправить SMS через шлюз»). Если шлюз недоступен, сайт показывает понятное сообщение «попробуйте позже», а процесс заканчивается финальной ошибкой. Пользователь в базе не создаётся.

Вывод

BPMN помогает просто и наглядно показать, кто и что делает при регистрации. Лучше всего на диаграмме видно проверку полей, блокировку кнопки и то, что происходит при ошибках. Фронтенд-разработчику это помогает понять, как должна вести себя форма. Бэкенд-разработчику — какие проверки и запросы нужны на сервере. UX-аналитику — увидеть, что каждое требование привязано к конкретной задаче и не забыто. Ещё один плюс: диаграмму можно открыть в редакторе BPMN и поправить.